

1 Key Concepts

1.1 Formal Languages

Definition 1 (alphabet). An *alphabet* is a *nonempty, finite* set of symbols.

Definition 2 (string). A *string* over an alphabet Σ is a *finite, ordered* sequence of symbols from Σ .

The set of *all* (finite-length) strings over an alphabet Σ is denoted Σ^* . (The “star” superscript $*$ is known as the Kleene star operator.)

Definition 3 (language). A *language* L over an alphabet Σ is a subset of Σ^* , i.e., $L \subseteq \Sigma^*$. In other words, it is a set of (finite-length) strings over Σ .

1.2 Finite Automata

Definition 4 (DFA). A *deterministic finite automaton* (DFA) is a five-tuple

$$M = (\Sigma, Q, q_0, F, \delta),$$

where:

- Σ is the (finite) input alphabet; an input to the automaton is any (finite) string over this alphabet (i.e., an element of Σ^*).
- Q is the finite set of the automaton’s states.
- $q_0 \in Q$ is the initial state.
- $F \subseteq Q$ is the subset of accepting states. If, after processing the entire input string, the machine ends in one of these states, it is said to *accept* the string; otherwise, it is said to *reject* the string.
- δ is the transition function, which maps a state and input symbol to a next state:

$$\delta: Q \times \Sigma \rightarrow Q.$$

(Recall that $Q \times \Sigma$ is the set of all pairs whose first component is a state in Q , and whose second component is a symbol in Σ .) This function defines the automaton’s transitions as follows: if it is in state $q \in Q$ and reads the next input symbol $\sigma \in \Sigma$, then it transitions to the next state $\delta(q, \sigma) \in Q$. We emphasize that δ must be a *function*, i.e., for every state-symbol pair, there must be *exactly one* next state.

Definition 5 (Language of a DFA). The *language of a finite automaton* M is the set of strings that the automaton accepts: $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$. We also say that M *decides* this language: it accepts every string in $L(M)$, and rejects every string not in $L(M)$.

1.3 Turing Machines

Definition 6 (Turing Machine). A Turing machine is a seven-tuple

$$M = (\Sigma, \Gamma, Q, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}}, \delta)$$

whose components are as follows:

- Σ is the (finite) input alphabet. An input to the machine is any (finite) string over this alphabet (i.e., an element of Σ^*).
- Γ is the (finite) tape alphabet, which must contain Σ as a subset (i.e., $\Sigma \subseteq \Gamma$), in addition to a special blank symbol $\perp \in \Gamma$ that is not in the input alphabet (i.e., $\perp \notin \Sigma$). It may contain other elements as well. The alphabet Γ is the set of symbols that may be written to the cells of the tape.
- Q is the finite set of states.
- $q_{\text{start}} \in Q$ is the initial state.
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ are, respectively, the accept state and reject states, which must be distinct. Together, they comprise the set of final states $F = \{q_{\text{acc}}, q_{\text{rej}}\}$. If the machine ever transitions to either of these states, then the computation *immediately* halts, and the machine is said to *accept* or *reject*, respectively.
- δ is the transition function. Its input is a non-final state and a tape symbol (as read by the head), and its output is a new state, a new tape symbol (to be written by the head), and a direction for the head to move (left or right). Formally, it is a function

$$\delta: \underbrace{(Q \setminus F)}_{\text{Non-terminal state}} \times \underbrace{\Gamma}_{\text{Cell contents}} \rightarrow \underbrace{Q}_{\text{Next state}} \times \underbrace{\Gamma}_{\text{Symbol to write}} \times \underbrace{\{L, R\}}_{\text{Where to move the head}},$$

where L and R represent “left” and “right”, respectively.

A Turing machine M runs on an input x as follows. Initially, the characters of x appear in order from the beginning of the tape (one character per cell), and the tape head points to the first cell of the tape. The machine starts in the initial state q_0 . The machine then repeatedly applies its transition function (each time using its current state and the symbol read by the tape head to write a symbol, move the head, and transition to the next state), until it transitions to one of the final states (if it ever does so).

Unlike a DFA, the behavior of a Turing machine M when run on an input $x \in \Sigma^*$, denoted $M(x)$, falls into exactly one of *three* cases:

1. $M(x)$ eventually *accepts* (makes a transition to the accept state q_{acc});
2. $M(x)$ eventually *rejects* (makes a transition to the reject state q_{rej});
3. $M(x)$ runs forever, or “*loops*” (it never makes a transition to a final state).

Church-Turing Thesis. This posits that a function can be computed by an “effective” or “mechanical” procedure—in other words, an algorithm—if and only if it can be computed by some Turing machine. It is not a theorem, but a statement of belief (or an axiom) about what “computation” is, and that it is captured by the Turing-machine model.

Definition 7 (Language of a Turing machine). The *language* $L(M)$ of a Turing machine M is the set of strings that the machine accepts:

$$L(M) = \{x : M \text{ accepts } x\}$$

Definition 8 (‘Decides’ for Turing machines). A Turing machine M *decides* a language $L \subseteq \Sigma^*$ if:

1. M accepts every $x \in L$, and
2. M rejects every $x \notin L$.

Equivalently:

1. M halts (accepts or rejects) on every $x \in \Sigma^*$, and
2. $x \in L$ if and only if $M(x)$ accepts.

A language is *decidable* if some Turing machine decides it.

Definition 9 ('Recognizes' for Turing machines). A Turing machine M *recognizes* a language $L \subseteq \Sigma^*$ if:

1. M accepts every $x \in L$, and
2. M *rejects or loops* on every $x \notin L$.

Equivalently: $x \in L$ if and only if $M(x)$ accepts.

A language is *recognizable* if some Turing machine recognizes it.

2 Discussion Problems

2.1 Decision Problems and Languages

1. Determine whether each of the following problems is a decision problem. If it is, formulate a language containing exactly the “yes” instances of the problem (make sure to specify how you would encode the elements of the language). If it is not, explain how you could change it into a decision problem.

(a) Given a set of cities, find a set of three cities whose name share at least 3 letters.

- (b) Given a list of classes (specified by (s_c, e_c) where s_c is the start time of class c and e is the end time of class c), is it possible to enroll in at least three classes without having any overlaps in lecture times?

2. For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, and (iii) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.
- (a) Given a binary string b , does it have an even number of 0s?
 - (b) Does a given non-negative integer x , when written in base 10, have 3 as the last digit?
 - (c) Does a given C++ program, when run on input “eecs376”, halt within 376 steps?

2.2 DFAs

3. True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}.$

Justify your answer.

4. True or False: there is a DFA that decides the following language: $\{x : x \text{ is a palindrome}\}$, where the alphabet Σ has at least two elements. Justify your answer.

2.3 Turing Machines

Are the following true or false?

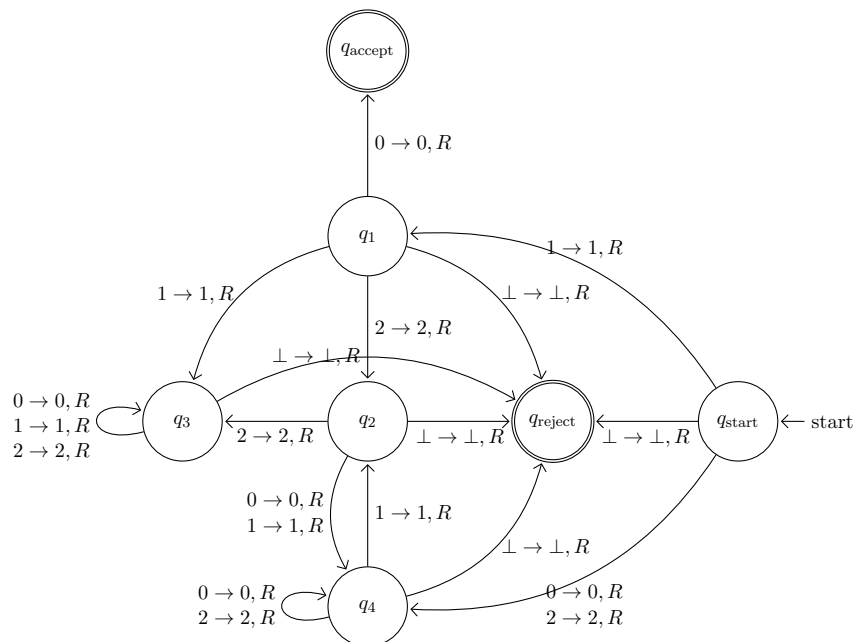
1. (True/False) A Turing machine can potentially decide more than one distinct language.

2. (True/False) For any decidable language L , there is more than one distinct Turing machine M that decides L .

3. (True/False) Any Turing machine recognizes some language.

4. (True/False) There exists a language X that is the language of some three-tape Turing machine, but is not the language of any two-tape Turing machine.

5. Consider the Turing Machine whose state diagram is given below:



Which of the following statements is true about this Turing Machine?

- (a) It accepts all strings that have the substring "10."
- (b) It accepts all strings that start with the substring "10."
- (c) It loops on any input string consisting only of 2s.
- (d) The string "10⊥1" is in the language of this Turing Machine.

2.4 Decidability

1. Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

3 Additional Problems

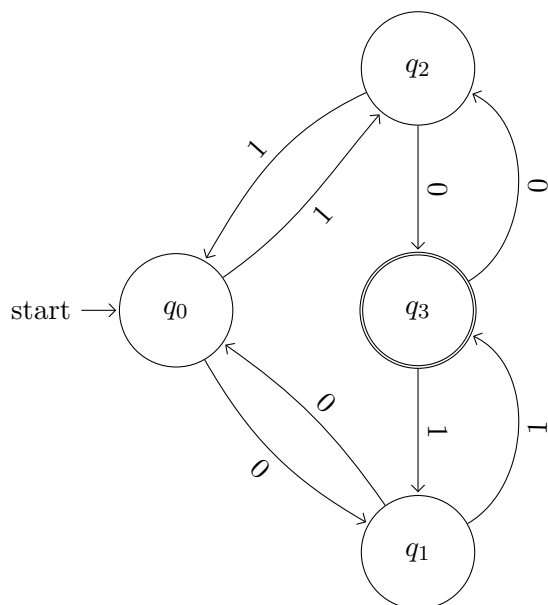
3.1 Decision Problems and Languages

1. The language that represents all positive integers divisible by 4 can be written as:

- (a) $L = \{n \in \mathbb{Z}^+ : n \bmod 4 = 0\}$
- (b) $L = \{n, k \in \mathbb{Z} : n/4 = k\}$
- (c) $L = \{n \in \mathbb{Z}^+ : \gcd(n, 4) = 1\}$
- (d) $L = \{n, k \in \mathbb{Z}^+ : n \bmod 4 = 2\}$
- (e) None of the above

3.2 DFAs

1. Consider the following DFA:

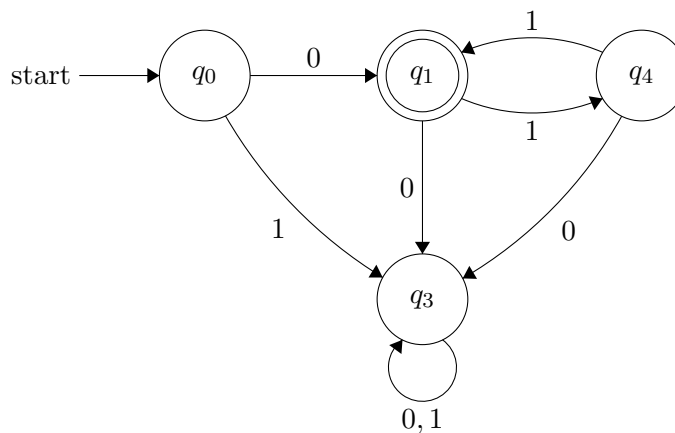


- (a) Recall that a DFA is formally defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$. State what these components are for the above DFA.

- (b) What language does the above DFA decide?

2. Draw a DFA that decides the language 1110^* .

3. Consider the following DFA, where the alphabet is $\Sigma = \{0, 1\}$.

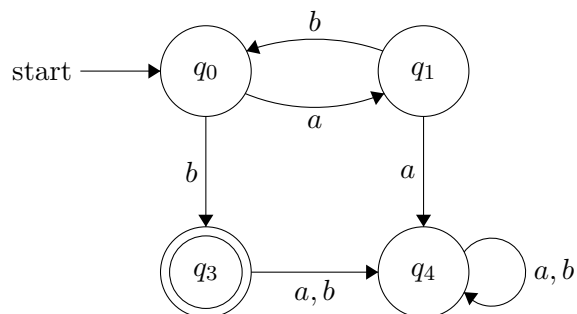


(a) For each of the following strings, determine whether the string is accepted by the DFA:

- ε
- 0
- 1
- 010
- 011
- 111

(b) What language does the DFA decide? Provide your answer in ‘string notation’.

4. Consider the following DFA, where the alphabet is $\Sigma = \{a, b\}$.

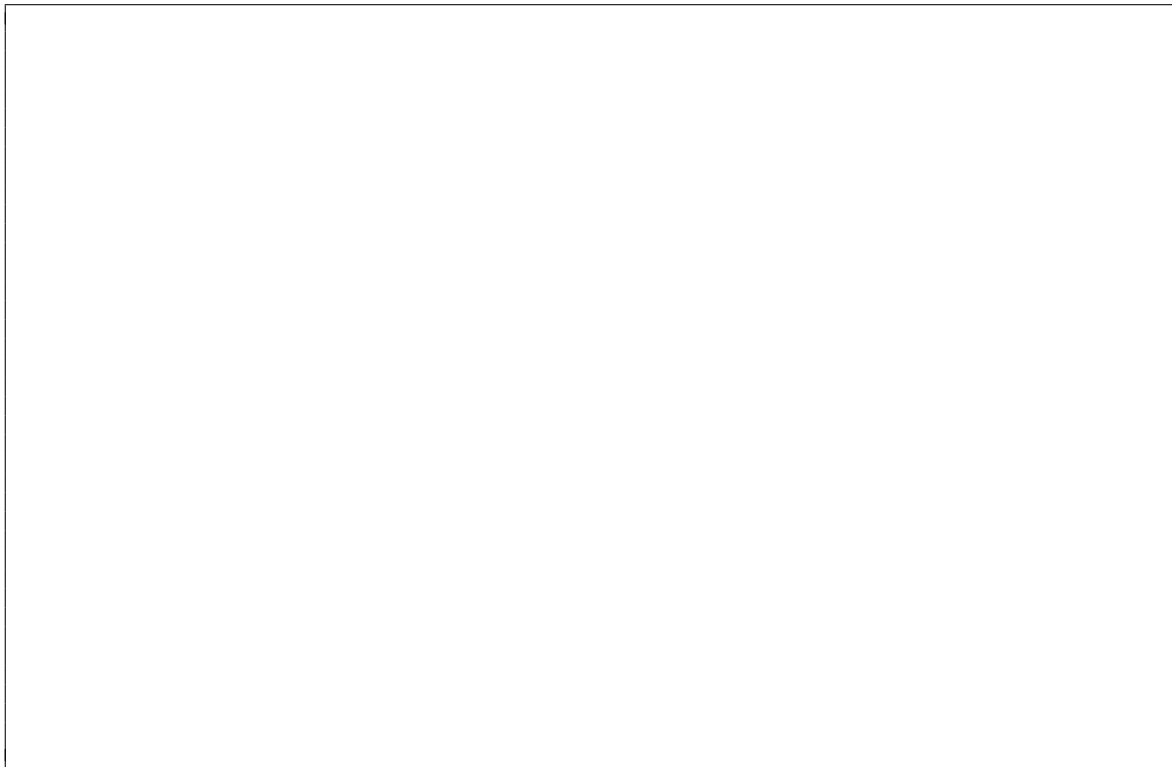


- (a) For each of the following strings, determine whether the string is accepted by the DFA:

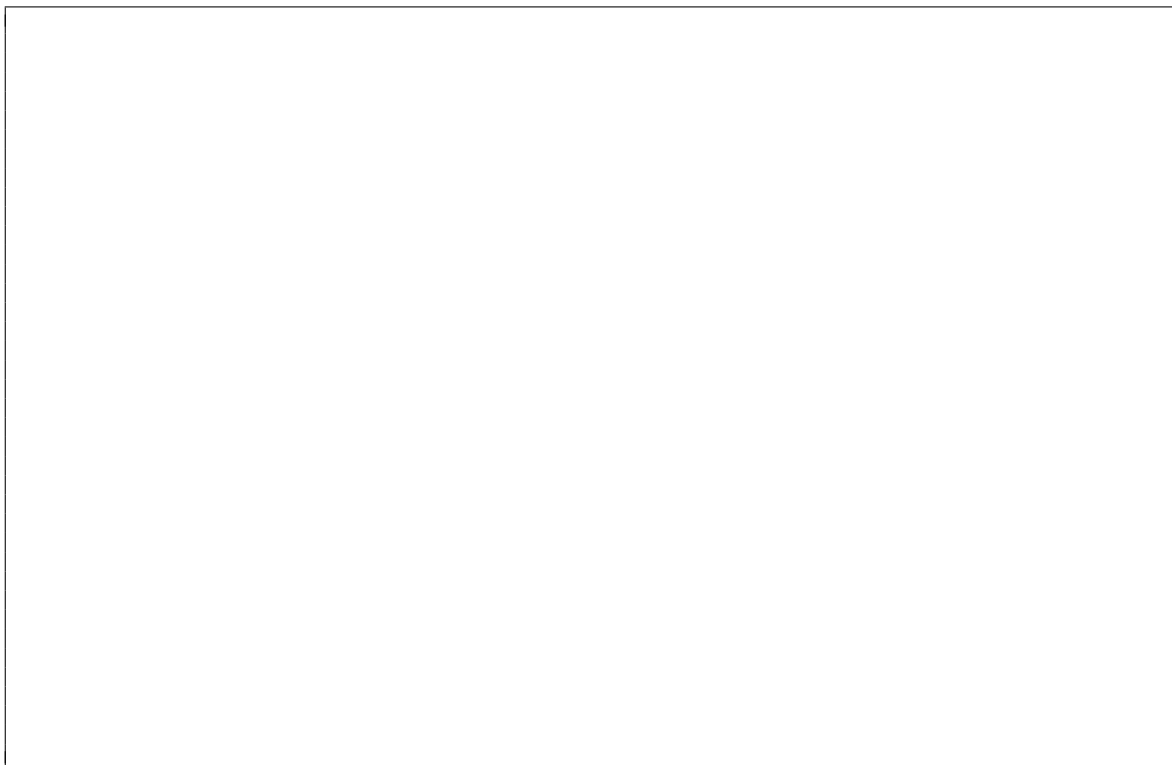
- ε
- a
- b
- bb
- aab
- $abbabb$
- $ababb$

- (b) What language does this DFA decide? Provide your answer in ‘string notation’.

5. Draw a DFA that decides the language ab^*ab .



6. Draw a DFA that decides the language $ab^*(cbc)^*aa^*$.



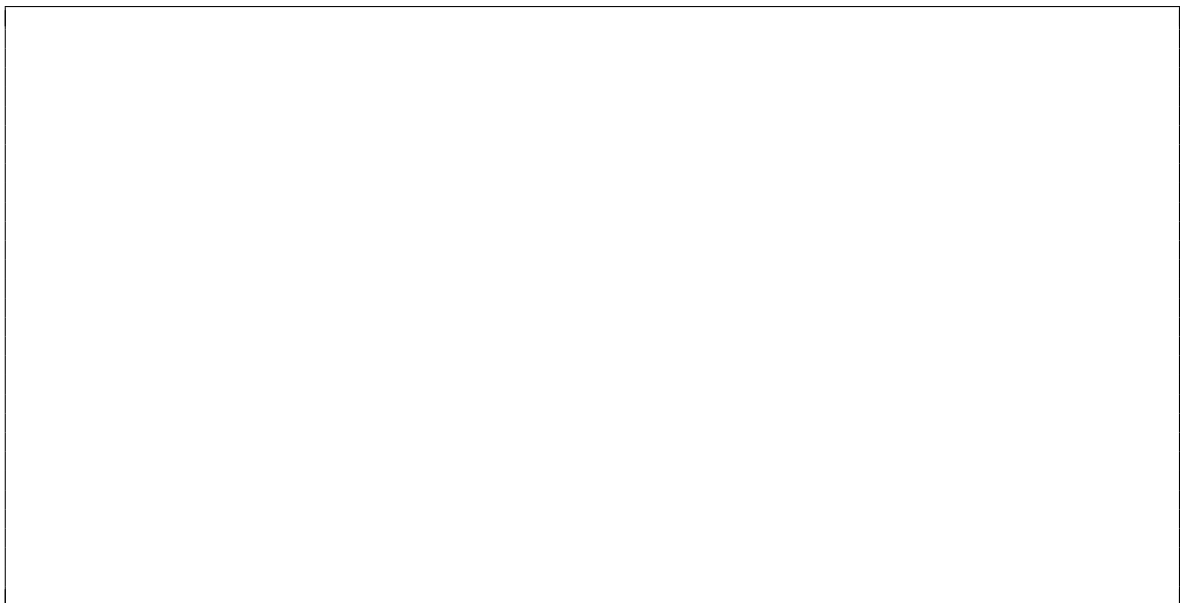
7. Draw a DFA that decides the language

$\{x \in \{0, 1\}^* : x \text{ has an even number of 0s and the last character of } x \text{ is } 1\}.$



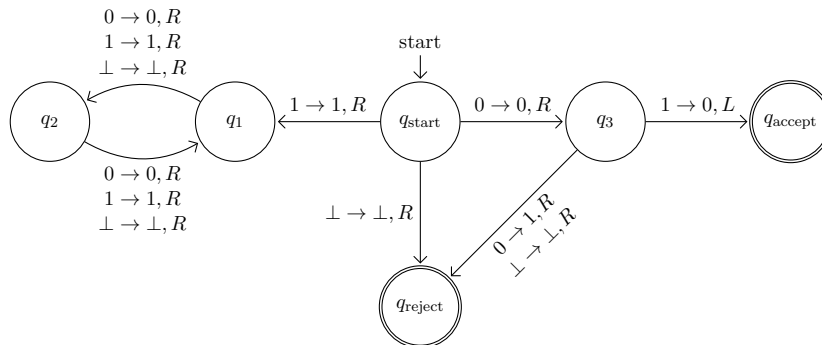
8. Draw a DFA that accepts only the set of binary strings of length $3n$, for some integer $n \geq 0$, that contain exactly two 0's for every "block" of 3 characters.

For example, 100010 is in the language because it has two blocks, 100 and 010, which both have *exactly* two 0's. The empty string, ϵ , is also (vacuously) in the language. On the other hand, 00100 is not in the language because its length is not a multiple of 3 so it cannot be split into blocks, and 110001 is not in the language because there is only one 0 in its first block, 110.



3.3 Turing Machines

1. Consider the Turing Machine whose state diagram is given below:



What language over $\Sigma = \{0, 1\}$ does this Turing machine decide, if any?

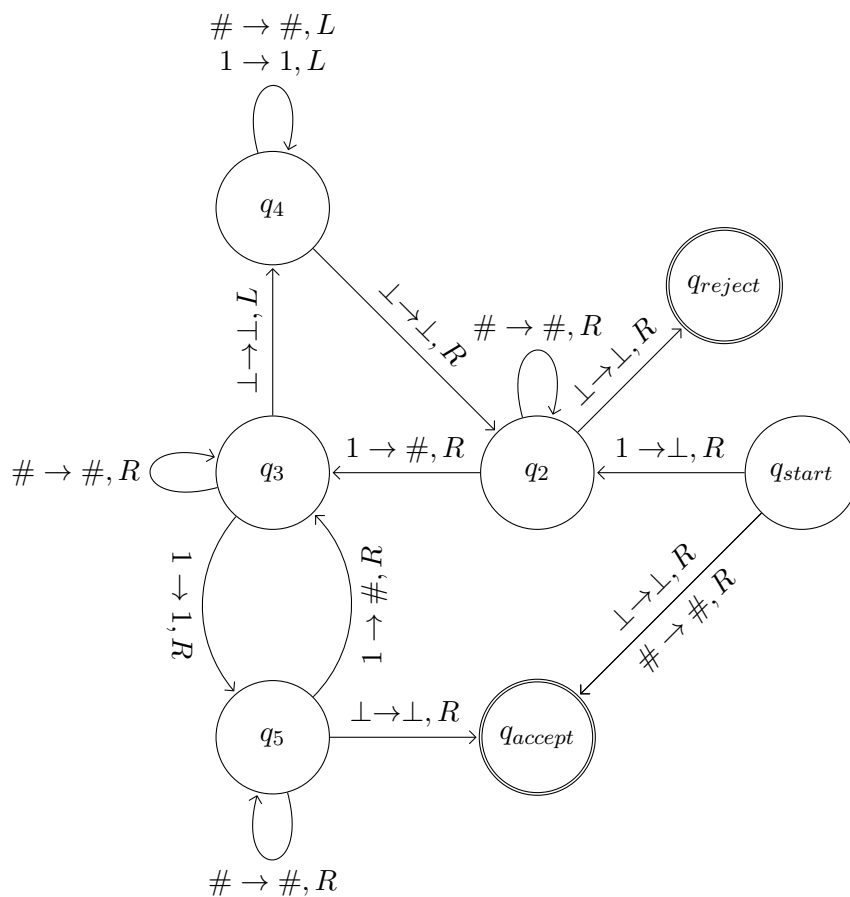
Note: We say that M decides language A if $A = \{w : M \text{ accepts } w\}$ and M halts on all inputs.

- (a) $\{01\} \cup \{s \in \Sigma^* : s \text{ starts with } 1\}$
- (b) $\{01\}$
- (c) $\{s \in \Sigma^* : s \text{ starts with } 1\}$
- (d) $\{\varepsilon, 0, 00\}$
- (e) None of the above

2. (Every / Some / No) Turing machine writes to only finitely many of its tape cells during its execution.

3. (Every / Some / No) Turing machine that decides a language writes to only finitely many of its tape cells during its execution.

4. Consider the Turing machine represented by the following figure, for input alphabet $\Sigma = \{1\}$ and tape alphabet $\Gamma = \{1, \#, \perp\}$. (Note that there are *two* extra characters in the latter.)



What language does this Turing machine decide, if any? Justify your answer.

Hint: What strings does it not accept?

3.4 Decidability

1. Show the following statement is true: For any decidable language L , the language $L' = L \cup \{\varepsilon\}$ is decidable.

2. The set difference of S and T is written $S \setminus T$, and defined as $S \setminus T = \{s \in S : s \notin T\}$.
If S and T are both decidable, $S \setminus T$ is (always/sometimes/never) decidable.